Advanced Logic Circuit Design
Exercise 5

Hiroshi Saito, Yoichi Tomioka

The University of Aizu

{hiroshis, ytomioka}@u-aizu.ac.jp

Exercise 5

- Purpose
  - To model, synthesize, and evaluate a MIPS processor with the single cycle execution
- About MIPS processor
  - Described in the textbook “Computer Organization and Design”
  - Single cycle execution for instructions
  - Supports 9 instructions

MIPS Processor
Supported Instructions

- **R-type instructions**
  - **add (addition)** — **e.g., add $t1, $s2, $s3 ($s2 + $s3 → $t1)**
  - **sub (subtraction)** — **e.g., sub $t1, $s2, $s3 ($s2 - $s3 → $t1)**
  - **and** — **e.g., and $t1, $s2, $s3 ($s2 & $s3 → $t1)**
  - **or** — **e.g., or $t1, $s2, $s3 ($s2 | $s3 → $t1)**
  - **slt (set on less than)** — **e.g., slt $t1, $s2, $s3 (if $s2 < $s3 then $t1 = 1 else $t1 = 0)**

$t1 - destination address, $s2 and $s3 - source addresses,
op – opcode, shamt – shift amount, funct – function

31 2625	2120 1615	1110 6 5	0
op	rs ($s2)	rt ($s3)	rd ($t1)	shamt	funct

add $t1, $s2, $s3	000000	10010	10011	10001	00000	100000
sub $t1, $s2, $s3	000000	10010	10011	10001	00000	100010
and $t1, $s2, $s3	000000	10010	10011	10001	00000	100100
or $t1, $s2, $s3	000000	10010	10011	10001	00000	100101
slt $t1, $s2, $s3	000000	10010	10011	10001	00000	101010

MIPS Processor
Supported Instructions

- **Instructions for memory access**
  - **lw (load)** **e.g., lw $t1, 100($s2) (mem[$s2 + 100] → $t1)**
  - **sw (store)** **e.g., sw $t1, 100($s2) ($t1 → mem[$s2 + 100])**

    Here, 100 is a decimal.

31 – 26	25 – 21	20 – 16	15 – 0
op	rs ($s2)	rt ($t1)	address offset (100)

Address offset 100 is obtained by memory index * 4
E.g., a[25] → 25 is index. Therefore, offset becomes 25 * 4 = 100

lw $t1, 100($s2) 100011 10010 10001 0000000001100100
sw $t1, 100($s2) 101011 10010 10001 0000000001100100

MIPS Processor
Supported Instructions

- **Branch instruction**
  - **beq (branch equal)** **e.g., beq $s1, $s2, L (if $s1 == $s2, go to L)**

31 2625 2120 1615 0

op	rs ($s1)	rt ($s2)	address offset (L / 4)

beq $1, $2, 100 (L is decimal 100) 000100 10001 10010 0000000000011001

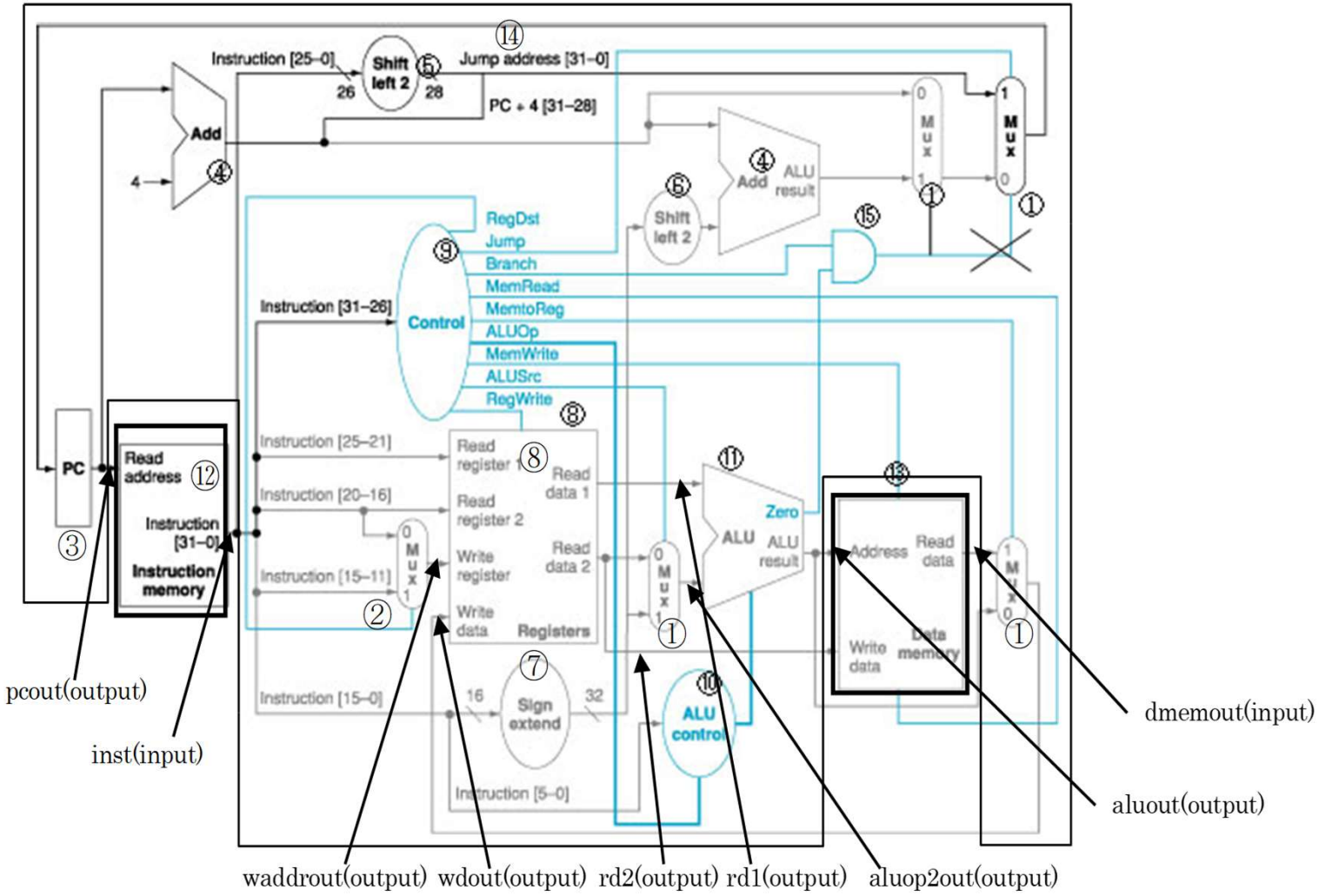
- **Jump instruction**
  - **j (jump) – e.g., j 10000**

31 2625 0

op	address offset (L / 4)

j 10000 (L is decimal 10000) 000010 00000000000000100111000100

MIPS Processor Architecture



Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

1. Create a project named “mips” (Device: Cyclone IV EP4CE115F29C7)
2. Modeling of the 2-input 32-bit multiplexer (①) by Verilog HDL (mux2_32.v)
   - You can use the Verilog HDL model in exercise 3.1 directly
3. Modeling of the 2-input 5-bit multiplexer (②) by Verilog HDL (mux2_5.v)
   - You need to modify the bit-width of the Verilog HDL model in exercise 3.1 (from 32 to 5)
4. Modeling of 32-bit adder (④) by Verilog HDL (add32.v)
   - You can use the Verilog HDL model in exercise 3.2 directly

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

5. Modeling of the program counter (③) by Verilog HDL (reg32a.v)
   1. Create a project named “reg32a” (Device: Cyclone IV EP4CE115F29C7)
   2. Modeling by Verilog HDL (reg32a.v)
      - As this is a 32-bit register with an asynchronous reset, you need to modify the Verilog HDL model reg32s.v in exercise 3.4
   3. RTL simulation
      1. Prepare a test bench (reg32a_tb.v)

time	reset	in
0	0	32’h00000###
5	1	32’h00000###
15	0	32’h00000###
55	0	32’h0000****

- the first 3 digits of your ID
**** - the last 4 digits of your ID

      2. Simulate the test bench assigning clock cycle time as 20 ns and total simulation time as 100 ns
      3. Print the waveform and explain the correctness of reg32a on the waveform

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

6. Modeling of 2-bit left shifter (⑤) by Verilog HDL
   1. Create a project named “left2_28” (Device: Cyclone IV EP4CE115F29C7)
   2. Modeling by Verilog HDL (left2_28.v)
      - Input: in [25:0]
      - Output: out [27:0]
      - Function: out is {in, 2’b00}
   3. RTL simulation
      1. Prepare a test bench (left2_28_tb.v)
      2. Assign your id to “in” in decimal as a test input (you can assign arbitrary simulation time)
      3. Simulate the test bench
      4. Print the waveform and explain the correctness of left2_28 on the waveform
         - Show in and out in binary

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

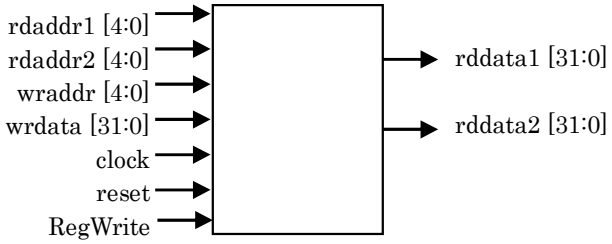
7. Modeling of 2-bit left shifter (⑥) by Verilog HDL
    1. Create a project named “left2_32” (Device: Cyclone IV EP4CE115F29C7)
    2. Modeling by Verilog HDL (left2_32.v)
        - Input: in [31:0]
        - Output: out [31:0]
        - Function: out is {in[29:0], 2’b00}
    3. RTL simulation
        1. Prepare a test bench (left2_32_tb.v)
        2. Assign your id to “in” in decimal as a test input (you can assign arbitrary simulation time)
        3. Simulate the test bench
        4. Print the waveform and explain the correctness of left2_32 on the waveform
            - Show in and out in binary

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

8. Modeling of sign extension (⑦) by Verilog HDL
   1. Create a project named “sign16_32” (Device: Cyclone IV EP4CE115F29C7)
   2. Modeling by Verilog HDL (sign16_32.v)
      - Input: in [15:0]
      - Output: out [31:0]
      - Function:
        - out = {in[15], in[15], in[15], in[15], in[15], in[15], in[15], in[15], in[15], in[15], in[15], in[15], in[15], in[15], in[15], in[15], in};
   3. RTL simulation
      1. Prepare a test bench (sign16_32_tb.v)
      2. Specify two test inputs to “in” (you can assign arbitrary simulation time)
         1. 0000 1001 0011 1001
         2. 1111 0000 1100 0011
      3. Simulate the test bench
      4. Print the waveform and explain the correctness of sign16_32 on the waveform
         - Show in and out in binary

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

9. Modeling of register file (⑧) by Verilog HDL



- This register file consists of 32 32-bit registers with an asynchronous reset
- Two data can be read and one data can be written within a clock cycle

1. Create project "regfile32_32" (Device: Cyclone IV EP4CE115F29C7)
2. Modeling by Verilog HDL (regfile32_32.v)
   - Declaration of 32 32-bit registers
     - reg [31:0] register [31:0]
   - Functions
     - Reading from register file
       - According to the value of "rdaddr1", the content of corresponding register is read from "rddata1"
       - According to the value of "rdaddr2", the content of corresponding register is read from "rddata2"
     - Writing to register file
       - According to the value of "wraddr", "wrdata" is written into corresponding register when a positive edge of "clock" happens and RegWrite = 1

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

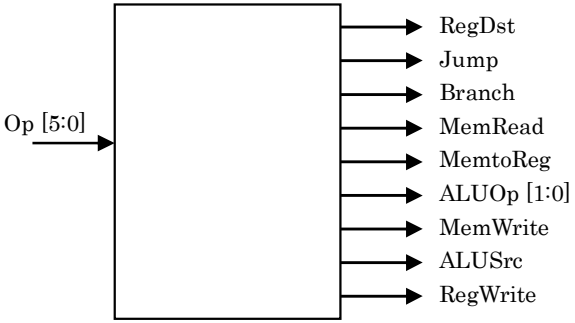
3. **RTL simulation**
   1. **Prepare a test bench (regfile32_32_tb.v)**
   2. **Simulate the following test patterns**
      - **clock cycle time = 10 ns**

Time from 0 [ns]	Input signal changes
3	reset = 1
8	reset = 0
16	RegWrite = 1, wraddr = 5'd1, wrdata = 32'd100
26	wraddr = 5'd4, wrdata = 32'd10
36	wraddr = 5'd8, wrdata = 32'd777
46	wraddr = 5'd16 wrdata = 32'd9999
56	RegWrite = 0, rdaddr1 = 5'd1
66	rdaddr2 = 5'd4
76	rdaddr1 = 5'd8, rdaddr2 = 5'd16
86	rdaddr1 = 5'd24

   3. **Print the waveform and explain the correctness of reg32_32 on the waveform**
      - **Show rdaddr1, rdaddr2, wraddr, wrdata, rddata1, and rddata2 in decimal**

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

10. Modeling of controller (decoder) (⑨) by Verilog HDL



	R-type inst.	Lw	Sw	Beq	j
Op[5]	0	1	1	0	0
Op[4]	0	0	0	0	0
Op[3]	0	0	1	0	0
Op[2]	0	0	0	1	0
Op[1]	0	1	1	0	1
Op[0]	0	1	1	0	0
RegDst	1	0	*	*	0
ALUSrc	0	1	1	0	0
MemtoReg	0	1	*	*	0
RegWrite	1	1	0	0	0
MemRead	0	1	0	0	0
MemWrite	0	0	1	0	0
Branch	0	0	0	1	0
ALUOp [1]	1	0	0	0	0
ALUOp [0]	0	0	0	1	0
Jump	0	0	0	0	1

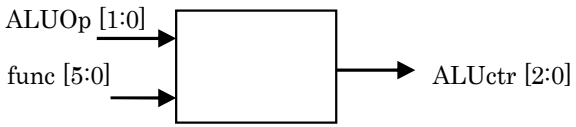
1. Create a project named “ctr” (Device: Cyclone IV EP4CE115F29C7)
2. Modeling by Verilog HDL (ctr.v)

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

3. RTL simulation
   1. Prepare a test bench (ctr_tb.v) to simulate five cases of Op [5:0] specified in the turth table (you can assign arbitrary simulation time)
   2. Simulate the test bench
   3. Print the waveform and explain the correctness of ctr on the waveform
      - Show all signals in binary

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

11. Modeling of ALU control (decoder) (⑩) by Verilog HDL

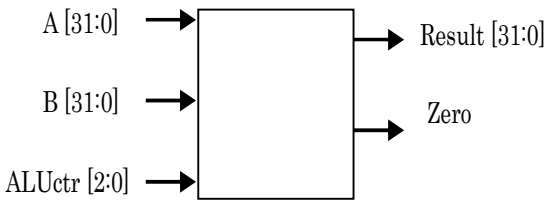


ALUOp [1]	ALUOp [0]	func [5]	func [4]	func [3]	func [2]	func [1]	func [0]	ALUCtr [2:0]
0	0	*	*	*	*	*	*	010
0	1	*	*	*	*	*	*	110
1	0	*	*	0	0	0	0	010
1	0	*	*	0	0	1	0	110
1	0	*	*	0	1	0	0	000
1	0	*	*	0	1	0	1	001
1	0	*	*	1	0	1	0	111

1. Create a project named (Device: Cyclone IV EP4CE115F29C7)
2. Modeling by Verilog HDL (aluctr.v)
3. RTL simulation
    1. Prepare a test bench (aluctr_tb.v) to simulate 7 cases of ALUOp [1:0] and func [5:0] specified in the truth table (you can assign arbitrary simulation time)
    2. Simulate the test bench
    3. Print the waveform and explain the correctness of aluctr on the waveform
        - Show all signals in binary

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

12. Modeling of ALU (⑪) by Verilog HDL



ALUctl [2:0]	Function
000	AND
001	OR
010	ADD
110	SUB
111	Set on less than

"Zero" becomes 1 when the output signal of ALU "Result" is 0

1. **Create a project named "alu" (Device: Cyclone IV EP4CE115F29C7)**
2. **Modeling by Verilog HDL (alu.v) based on exercise 3.3**

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

3. RTL simulation
   1. Prepare a test bench (alu_tb.v) using the following table (you can assign arbitrary simulation time)

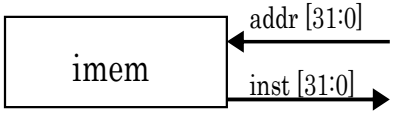
in1	in2	ALUctr
32'd100	32'd***	000
32'd100	32'd***	001
32'd100	32'd***	010
32'd100	32'd####	110
32'd100	32'd####	111
32'd100	32'd####	110

- the first 3 digits of your ID
**** - the last 4 digits of your ID

   2. Simulate the test bench
   3. Print the waveform and explain the correctness of alu on the waveform
      - Show in1, in2, and Result in decimal

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

13. Modeling of instruction memory (⑫) by Verilog HDL



Definition for memory cells

reg [31:0] mcell [1023:0];

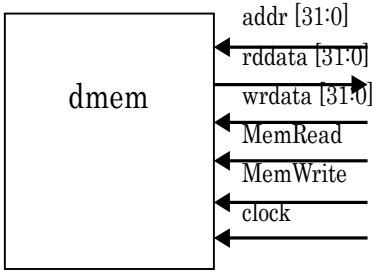
Note that write operation is not required for "imem"

1. Create a project named "imem" (Device: Cyclone IV EP4CE115F29C7)
2. Modeling by Verilog HDL (imem.v) based on the register file
3. RTL simulation
   1. Prepare imem_tb.v as follows
   2. Simulate the test bench
   3. Print the waveform and explain the correctness of imem on the waveform
      - Show addr and inst in decimal

```
imem i (.addr(addr), .inst(inst));
initial begin
    i.mcell[32'd0] = 32'h00000000;
    i.mcell[32'd1] = 32'h00000***; //*** first 3 digits of your id
    i.mcell[32'd10] = 32'h0000####; //#### last 4 digits of your id
    addr = 32'd0;
    #10
    addr = 32'd1;
    #10
    addr = 32'd10;
    #10
    addr = 32'd100;
    #10
    $finish;
end
```

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

14. Modeling of instruction memory (⑬) by Verilog HDL



Definition for memory cells

reg [31:0] mcell [1023:0];

Data are read from mcell [addr] whenever MemRead and addr are changed and MemRead = 1

Data are written into mcell [addr] by positive edge of clock signal when MemWrite = 1

1. **Create a project named “dmem” (Device: Cyclone IV EP4CE115F29C7)**
2. **Modeling by Verilog HDL (dmem.v) based on the register file**

Exercise 5.1 Modeling of the MIPS Processor based on the Bottom-up Design

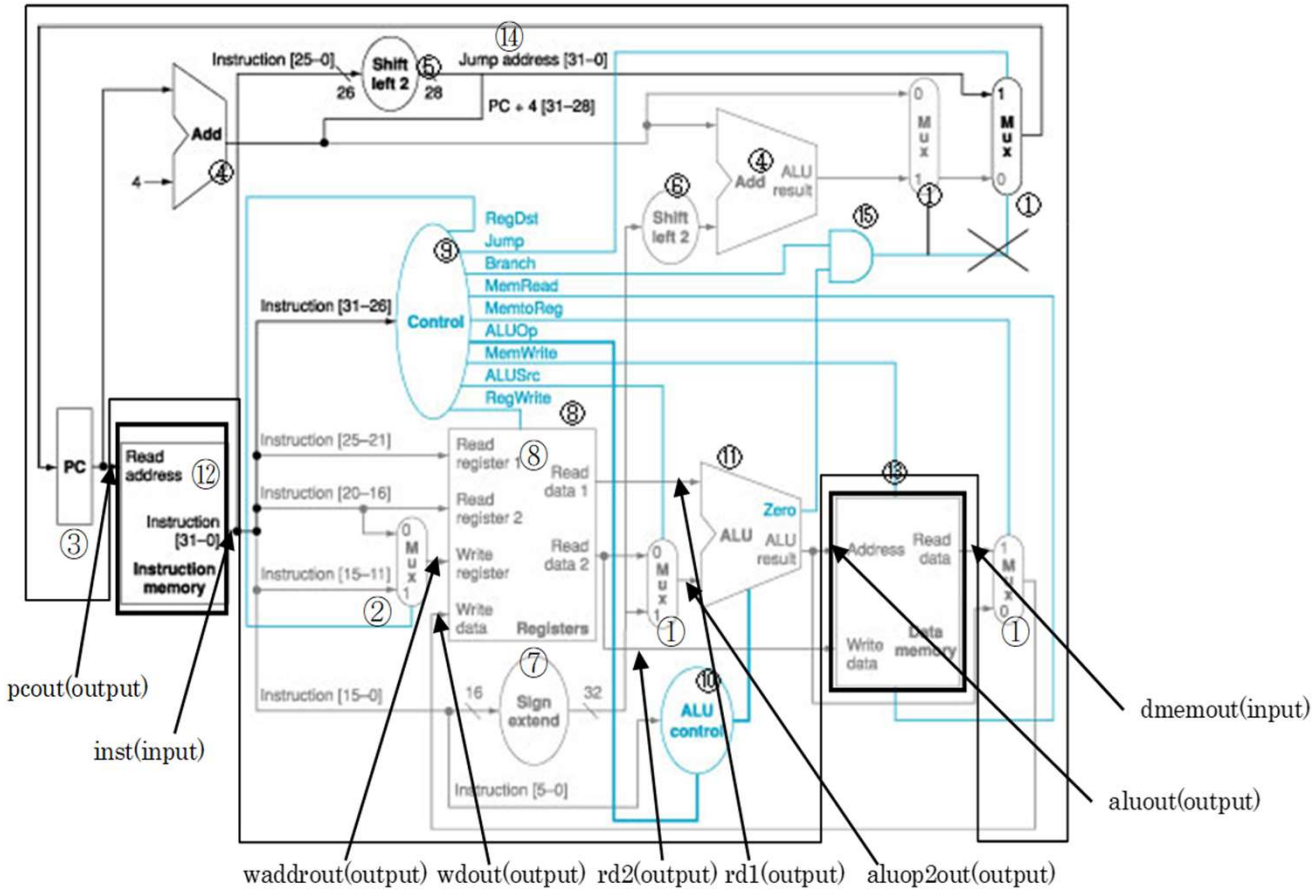
3. **RTL simulation**
   1. **Prepare dmem_tb.v as follow**

```
dmem d (.addr(addr), .rddata(rddata), .wrdata(wrdata), .MemRead(MemRead),
            .MemWrite(MemWrite), .clock(clock));
initial begin
    clock = 1'b0;
    MemRead = 1'b0;
    MemWrite = 1'b0;
    #6
    addr = 32'd10;
    MemWrite = 1'b1;
    wrdata = 32'd0########; //######## your id
    #10
    MemWrite = 1'b0;
    MemRead = 1'b1;
    #10
    addr = 32'd100;
    #10
    $finish;
end
always #5 clock = ~clock;
```

   2. **Simulate the test bench**
   3. **Print the waveform and explain the correctness of dmem on the waveform**
      - **Show addr, rddata, wrdata in decimal**

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

15. Modeling of the top-level module



- **Signals such as "pcout" should be declared as inputs or outputs to check signal flow during debugging**

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

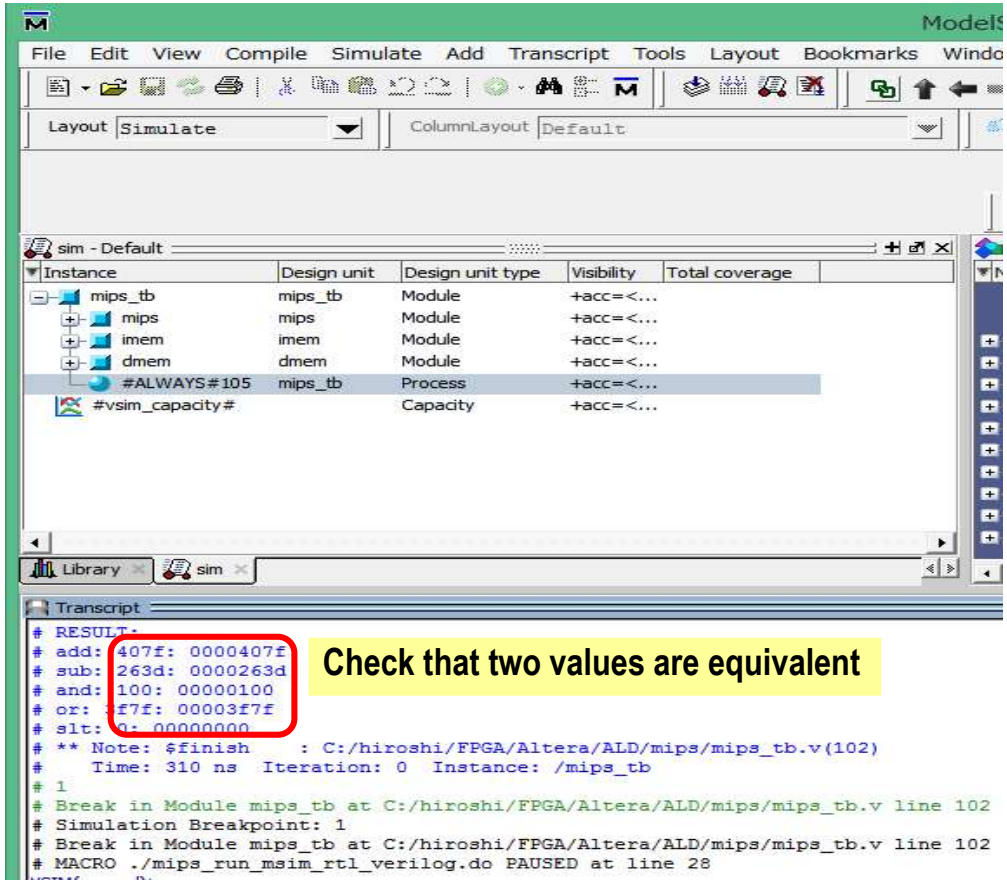
1. Modeling by Verilog HDL (mips.v)
   - Prepare in the project “mips”
   1. Instantiate all designed modules from 1.1 to 1.14 except “imem” and “dmem”
      - DON’T add imem.v and dmem.v in the project “mips”
   2. Assign ⑭ and ⑮
      - ⑭ is represented as follows
      - jaddr = {plus4out[31:28], left2_28out};
      - plus4out and left2_28out represent the outputs of ④ and ⑤
   3. Specify signals in the figure of Architecture as inputs and outputs (this is to know the flow of signal values in the MIPS processor)
   4. Specify output signals in “ctr” and output signals in “aluctr” as output signals of “mips” (this is also to know the flow of signal values)

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

2. RTL simulation
   1. Warning messages generated by Analysis and Synthesis must be solve as much as possible
   2. Download mips_tb.v from the web page and add it to project “mips”
      - This test bench module simulates all instructions in the MIPS processor
   3. Simulate mips_tb.v
      - Recommended to show signals with multiple-bits in hexadecimal

Exercise 5.1 Modeling of MIPS Processor Based on Bottom-up Design

4. Confirm that the MIPS processor works correctly by checking that displayed two values are equivalent (If they are not equivalent, you need to debug mips.v by checking connectivity of signals referring to the waveform)



5. Print msim_transcript in “mips/simulation/modelsim”

Debugging

- You must confirm that all modules work correctly through RTL simulation of them
- The design errors of the top level module mainly come from the connections among modules
  - You should check the connectivity carefully
- You may also check the correctness on the waveform through the flow of signal values
  - This is the reason why control signals and several data-path signals such as “pcout” are assigned as inputs and outputs

Exercise 5.2 Compilation and Evaluation

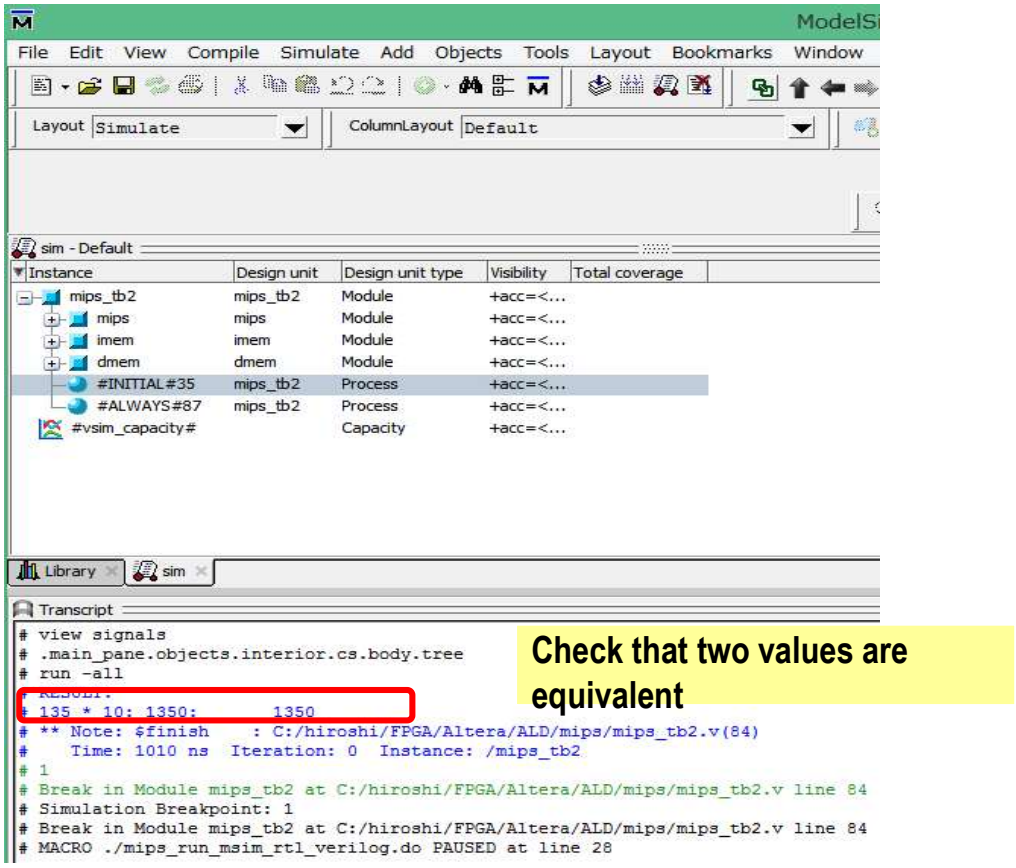
1. Compilation of the MIPS processor
   1. Remove mips_tb.v from project “mips”
   2. Compile mips.v assigning the clock cycle time constraint as 32 ns (use create_clock)
      - Note
        1. You need to prepare an SDC file (mips.sdc) with create_clock command
        2. You DON’T need to change setting
        3. Slacks for all timing models (e.g., slow 85℃) must be positive
   3. Report the number of logic elements and the number of registers from “Compilation Report”, and the worst-case delay for the slow 85℃ model

Exercise 5.2 Compilation and Evaluation

2. Gate level simulation of the MIPS processor
   1. Download mips_tb2.v from web page and add it to project “mips”
      - This test bench module calculates a product
   2. Assign last three digits of your ID to “dmem.mcell['h0000]”
   3. Modify ### and ???? to last three digits of your ID and the product of #### * 10 in the line $display("### * 10: ????: %d",dmem.mcell['h000c]);
   4. Modify the clock cycle time of mips_tb2.v to the one obtained at 5.2.1.2 (you also need to modify #*** appropriately)
      - Note: If the gate level simulation fails even though no timing violation, you need to increase the clock cycle time a little (e.g., 36 ns)
        - You need to compile again using the modified clock cycle time
   5. Simulate mips_tb2.v

Exercise 5.2 Compilation and Evaluation

6. Confirm that the MIPS processor works correctly by checking that displayed two values are equivalent (If they are not equivalent, you need to debug mips.v by checking connectivity of signals referring to the waveform)



7. Print msim_transcript in “mips/simulation/modelsim”

Exercise 5.2 Compilation and Evaluation

3. **Performance analysis**
   - **Report the execution time of your application based on the following equation or on the waveform**
     - **exectime = the number of executed instructions * clock cycle time**

Report 1

- Deadline: 13:20 on July 28
  - Delay within 13:20 on July 31 → -10% from your score
  - After 13:20 on July 31 → 0
- -5% from your score for each period if you absent exercise classes on July 10, 14, and 17
- Prepare a PDF file including
  - Title page (course name, exercise number, your ID, date, and your name)
  - Verilog HDL for all modules from 5.1.1 to 5.1.15 and test bench modules from 5.1.5 to 5.1.15
  - Waveform for RTL simulation with explanation from 5.1.5 to 5.1.14
  - Divide waveforms into several pages if characters are unclear or invisible
  - msim_transcript for mips_tb.v
- Screenshots are accepted only when some explanation is described in the screenshots (score is not counted if not)
  - Photos are not accepted
- Write in English if you put comments or explanations
- Submit the PDF file to the Moodle site

Report 2

- Deadline: 18:00 on Aug 1
  - After 18:00 on Aug 1 → 0
  - -5% from your score for each period if you absent exercise classes on July 24 and July 28
- Prepare a PDF file including
  - Title page (course name, exercise number, your ID, date, and your name)
  - SDC file
  - The number of logic elements, the number of registers, and the worst-case delay (with the print of “Compilation Report")
  - mips_tb2.v
  - msim_transcript for mips_tb2.v
  - Report for the execution time
- Screenshots are accepted only when some explanation is described in the screenshots (score is not counted if not)
  - Photos are not accepted
- Write in English if you put comments or explanations
- Submit the PDF file to the Moodle site

Recommended Schedule

- Follow as much as possible

7/11		7/14		7/17		7/24		7/28	
6	7	6	7	6	7	6	7	6	7
Intro.	5.1.1～ 5.1.8	5.1.9	～ 5.1.12	5.1.13	～ 5.1.15	Report1	5.2	5.2	Report 2